软件需求工程——完整知识点整理

> 课程：软件需求工程 | 院校：西北工业大学 软件学院

第一章 绪论：需求工程与软件需求

1.1 需求工程发展历程

1. **计算机发展初期**：软件规模不大，侧重代码编写
2. **软件危机爆发**：催生软件生存周期概念，需求分析成为第一阶段
3. **软件规模扩大**：需求分析越来越重要，需求分析活动贯穿整个生存周期
4. **80年代中期**：需求工程成为研究热点
5. **90年代**：Springer-Verlag发行《Requirements Engineering》；1993年ISRE国际研讨会；1994年ICRE国际会议；1996年RENOIR成立；2000年形成软件工程子领域

1.2 需求工程概念

1.2.1 需求工程的定义

- **定义**：需求工程是指应用已证实有效的技术、方法进行需求分析，确定客户需求，帮助分析人员理解问题并定义目标系统的所有外部特征的一门学科
- **特点**：通过合适的工具和记号系统地描述待开发系统及其行为特征和相关约束，形成需求文档，并对用户不断变化的需求演进给予支持
- **分类**：系统需求工程和软件需求工程

1.2.2 需求工程的概念

- 软件需求工程是把系统需求分解成子系统任务，分配给软件，通过反复分析、设计、比较研究、原型开发将系统需求转换成软件需求描述和性能参数
- 需求工程是一个不断反复的需求定义、文档记录、需求演进的过程，最终在验证基础上冻结需求

1.2.3 需求工程的阶段

Herb Krasner 五阶段生命周期：

1. 需求定义和分析
2. 需求决策
3. 形成需求规格
4. 需求实现与验证
5. 需求演进管理

Jarke & Pohl 三阶段周期法：获取 → 表示 → 验证

综合五阶段：

1. **需求获取**：与用户交流、观察系统、分析任务，开发、捕获和修订用户需求
2. **需求建模**：为系统建立概念模型，作为对需求的抽象描述，捕获现实世界语义
3. **形成需求规格**：生成需求模型构件的精确形式化描述，作为用户和开发者之间的协约

4. **需求验证**：以需求规格说明为输入，通过符号执行、模拟或快速原型等途径分析正确性和可行性
5. **需求管理**：支持系统需求演进，如需求变化和可跟踪性问题

1.3 软件需求的定义

1.3.1 软件需求的层次性

- 客户所定义的"需求"对开发者似乎是较高层次的产品概念
- 开发人员所说的"需求"对用户又像是详细设计
- 软件需求包含多个层次，不同层次从不同角度和不同程度反映细节

1.3.2 需求的多种解释

1. **Jones(1994)**：需求是用户所需要的并能触发一个程序或系统开发工作的说明
2. **Alan Davis(1993)**：从系统外部能发现系统所具有的满足于用户的特点、功能及属性等；强调产品是什么样的，而非怎样设计构造
3. **系统特性角度**：需求是指明必须实现什么的规格说明，描述系统的行为、特性或属性，是在开发过程中对系统的约束

1.3.3 IEEE定义（1997）

1. 用户为解决某个问题或达到某种目标而需具备的条件或能力（Capability）
2. 系统或系统部件要满足合同、标准、规范或其它正式规定文档而必须满足的条件或必须具备的能力
3. 一种反映上述(1)或(2)所描述的条件或能力的文档说明

> 定义包括从用户角度的外部行为，以及从开发者角度的一些内部特性

1.3.4 本课程定义

软件需求是指用户对目标软件系统在功能、行为、性能、设计约束等方面的期望。

流程：用户期望 → 问题分析 → 问题建模 → 形成需求规格说明

需求分析的特点：

1. 以系统规格说明和项目规划为出发点，从软件角度检查与调整
2. 需求规格说明是软件设计、实现、测试直至维护的主要基础
3. 良好的分析活动有助于避免或尽早剔除早期错误，提高生产效率，降低开发成本

1.3.5 软件需求的层次和分类

三个层次：

1. **业务需求**：反映组织机构或客户对系统高层次的目标要求，记录在项目视图与范围文档中
2. **用户需求**：描述用户的目标或用户要求系统必须完成的任务，用用例(use case)文档、场景描述(scenario)和事件-响应表表达
3. **功能需求**：定义开发人员必须实现的软件功能，使用户能完成其任务，从而满足业务需求

各组成部分关系：

- 业务需求 → 业务规则 → 项目视图与范围文档
- 用户需求 → 质量属性 / 约束条件 / 其它非功能需求（接口）→ 用例文档
- 功能需求 → 系统需求 → 软件需求规格说明

1.4 需求工程的内容

1.4.1 需求工程的内容及层次分解

需求工程 = **需求开发** + **需求管理**

- 需求开发：需求获取 → 需求分析 → 编写规格说明 → 需求验证
- 需求管理：需求跟踪管理 → 需求变更管理

1.4.2 需求工程活动

需求工程中的活动一般是多次循环的形式：需求获取 → 需求分析 → 编写规格说明 → 需求验证（可回到需求获取重新更正）

1.4.3 需求获取

主要活动：

1. 确定需求开发过程
2. 编写项目视图和范围
3. 用户群分类
4. 选择产品代表
5. 建立核心队伍
6. 确定用例
7. 召开应用程序开发联席会议
8. 分析用户工作流程
9. 确定质量属性
10. 检查问题报告
11. 需求重用

主要任务：收集材料、定义项目视图和范围、选择信息来源、选择获取方法并执行、记录获取结果

可能出现的问题：捕获范围不足、缺乏计划性、获取对象不明确、缺乏科学性

1.4.4 需求分析

- 需求分析是需求开发的核心任务，是获取用户需求后的粗加工过程
- 通过修正错误、补充遗漏、消除不一致，获得用户对软件系统的真正需求

需求分析具体描述：

1. 是对业务的分析：对问题域进行研究，而非系统结构
2. 是一种分解活动：按职责划分主题域，分解为流程、业务活动（用例）、业务步骤
3. 是一种提炼和整合活动：将用户原始需求合并到业务活动中
4. 是一种规范化活动：发现矛盾和冲突，并予以协调解决

需求分析的主要任务：背景分析 → 确定系统边界 → 需求建模 → 需求细化 → 确定优先级 → 子系统建立 → 数据字典创建 → 绘制关联图 → 原型开发 → 需求协商

1.4.5 编写需求规格说明书

将需求分析结果文档化，阐述软件系统必须提供的功能和性能以及限制条件，是系统测试、用户文档、设计和编码的基础。

主要任务：定制文档模板 → 编写需求规格说明

1.4.6 需求验证

审查需求规格说明是否正确和完整地表达了用户对软件系统的需求。

主要任务：

1. 执行验证：同行评审
2. 问题修正：发现问题及时修正

验证标准：

1. 每条需求都准确反映用户意图
2. 需求集整体具有完整性和一致性
3. 文档组织方式和书写方式具有可读性和可修改性

1.4.7 需求管理

任务和活动：

1. 建立和维护需求基线
2. 建立需求跟踪信息（后向跟踪和前向跟踪）

1.4.8 需求评审

评审提出的需求变更、评估每项变更的可能影响，决定是否实施。

作用：

1. 以可控制方式将需求变更融入项目
2. 使当前项目计划与需求一致
3. 估计变更影响并协商新承诺
4. 跟踪需求状态及变更情况

1.4.9 需求开发与需求管理的界限

- 需求开发：客户/市场 → 需求分析 → 编写文档 → 评审商议 → 基准需求说明
- 需求管理：当前基线 → 需求变更过程 → 修正后基线 → 项目变更

1.5 需求在总体方案中的位置

1.5.1 软件的生存周期

问题定义和可行性研究 → 制定开发计划 → 需求获取、分析 → 规格说明及验证 → 设计 → 实现 → 测试 → 部署 → 维护

1.5.2 需求与其他软件项目过程的关系

SRS在开发、测试、质量保证、项目管理以及相关项目过程中都起重要作用：

- **制定项目计划**：作为输入/基础
- **开发过程**：工作产品可追溯到软件需求
- **用户编制文档过程**：作为基础
- **系统测试过程**：验证实现的正确性，作为参考
- **变更控制过程**：作为基线进行变更
- **项目跟踪和控制过程**：跟踪状态

1.6 导致发生不合格需求说明的情况

风险	描述
风险1	用户参与不多导致产品无法被接受
风险2	用户需求增加带来过度耗费和降低质量
风险3	模棱两可的需求说明导致时间浪费和返工
风险4	不必要的特性（画蛇添足）
风险5	过分简略的需求说明遗漏关键需求
风险6	忽略用户分类导致有的用户失望
风险7	不完善的需求说明使项目计划无法准确进行

模棱两可的需求：

- 诸多读者对需求说明产生不同理解
- 单个读者能用不止一个方式解释某个需求
- 模棱两可的需求使开发者浪费时间，测试者重做测试
- 需求方面的错误返工耗费开发预算的40%
- 解决方法：从不同角度审查需求

不必要的特性（画蛇添足）：开发人员增加需求规格说明中未涉及的新功能，徒耗时间和成本

过于精简的规格说明：客户不明白需求分析重要性，所作规格说明仅涉及产品概念

忽略用户分类：不同人使用不同特性，使用频繁程度、受教育程度和经验水平不同

不准确的计划：对需求分析缺乏理解导致过分乐观的估计

1.7 良好需求具有的特性

每项需求应具备的10个特性：

序号	特性	说明
1	完整性	将所要实现的功能描述清楚，提供设计实现所需的全部必要信息

序号	特性	说明
2	正确性	准确陈述要开发的功能；只有用户代表才能确定正确性
3	可行性	在已知系统和环境的权能和限制范围内可以实施；需软件开发人员参加
4	必要性	每项需求都能回溯至某项客户的输入
5	划分优先级	给每项需求分配实施优先级
6	无二义性	对所有读者只有一个明确统一的解释；解决方法：正规审查、编写测试用例、开发原型、用用户语言表达
7	可验证性	能通过设计测试用例或其它验证方法来检查
8	一致性	与其它软件需求或高层需求不相矛盾
9	可修改性	在必要时可修订SRS；每项需求独立标识，只出现一次
10	可跟踪性	能在需求与根源和设计元素、源代码、测试用例之间建立链接链

1.8 高质量需求过程带来的好处

1. 开发后期和维护阶段重做工作大大减少
2. 多方风险承担者通力合作
3. 更好了解市场，开发前了解比遭批评后意识到节约成本
4. 用户积极参与使产品更有吸引力，建立良好客户关系
5. 需求明确分配到各子系统，简化软硬件集成
6. 有效变更控制和影响分析降低需求变更负面影响
7. 清晰无二义性需求文档有利于系统测试

1.9 客户的需求观

客户十大权利

1. 要求分析人员使用符合客户语言习惯的表达
2. 要求分析人员了解客户系统的业务及目标
3. 要求分析人员编写软件需求规格说明
4. 要求得到需求工作结果的解释说明
5. 要求开发人员保持合作的职业态度
6. 要求开发人员对需求和产品实施提供建议
7. 描述产品易使用的特性
8. 调整需求，允许重用已有软件组件
9. 要求对变更代价提供真实可信的评估

10. 获得满足功能和质量要求的系统

客户十大义务

1. 给分析人员讲解业务及术语
2. 抽出时间清楚地说明并完善需求
3. 力求准确详细地说明需求
4. 需要时及时对需求做出决策
5. 尊重开发人员的成本估算和可行性分析
6. 对需求划分优先级
7. 评审需求文档和原型
8. 需求变更时马上联系开发人员
9. 遵照开发组织处理需求变更的过程
10. 尊重需求工程中开发人员采用的流程

1.10 需求工程的推荐方法

类别	方法
需求获取	编写项目视图与范围、确定需求开发过程、用户群分类、选择产品代表、建立核心队伍、确定使用实例、召开联席会议、分析用户工作流程、确定质量属性、检查问题报告、需求重用
需求分析	绘制关联图、创建开发原型、分析可行性、确定需求优先级、为需求建立模型、编写数据字典、应用质量功能调配
编写规格说明书	采用SRS模版、指明需求来源、为每项需求注上标号、记录业务规范、创建需求跟踪能力矩阵
需求验证	审查需求文档、依据需求编写测试用例、编写用户手册、确定合格标准
需求管理	确定变更控制过程、建立变更控制委员会、进行变更影响分析、跟踪影响工作产品的每项变更、跟踪需求状态、衡量需求稳定性、使用需求管理工具
知识技能	培训需求分析人员、培训用户代表和管理人员、培训应用领域开发人员、汇编术语
项目管理	选择合适的生存周期、确定需求基本计划、协商约定、管理需求风险、跟踪需求工作

第二章 需求获取

2.1 需求获取概述

2.1.1 需求的三个层次

1. **业务需求**：不应包括用户需求或使用实例
2. **用户需求**：对用户有价值的内容或让用户喜欢的内容
3. **功能需求**：所有功能需求应源于用户需求；同时需获取非功能需求（如质量属性）

> 来自不同的时间、不同的来源、不同的目标、不同的对象、不同的方式 → 需求文档

2.1.2 需求获取的主要活动

1. 确定需求开发过程
2. 编写项目视图和范围
3. 用户群分类
4. 选择产品代表
5. 建立核心队伍
6. 召开应用程序开发联席会议
7. 分析用户工作流程
8. 确定质量属性
9. 检查问题报告
10. 需求分类重用
11. 用例确定

2.1.3 需求获取的任务

5个主要任务：收集材料 → 定义项目视图和范围 → 选择信息来源 → 选择获取方法并执行 → 记录获取结果

可能造成的问题：获取对象不明确、缺乏计划性

2.1.4 需求获取的重点探讨内容

1. 确定需求开发过程
2. 编写项目视图和范围文档
3. 将用户群分类并归纳特点，为每个用户类选择产品代表
4. 确定用例
5. 确定质量属性和其它非功能需求

2.2 需求分析员

2.2.1 定义

需求分析员（业务分析员、系统分析员、需求工程师、需求经理）是对项目涉众的需求进行收集、分析、记录和验证等职责的主要承担者。

2.2.2 主要工作

1. 定义业务需求：确定项目涉众和用户类别
2. 获取需求：交谈、讨论会、文档分析、调查、业务流程分析
3. 任务分析、事件列表、同类产品分析、根据现有系统推导需求
4. 分析需求、编写需求规格说明书
5. 为需求建模、主持对需求的验证、管理需求

2.2.3 必备技能/知识/培养

技能	知识	培养途径
倾听、交谈和提问技巧	需求管理技术	从用户转为分析员
分析、协调、观察能力	软件开发生存周期相关技术	从开发人员转为分析员
写作、组织、建模能力	领域知识和技术	领域专家作为分析员
人际交往能力、创造力		

2.3 建立项目视图与范围

2.3.1 基本内容

- **项目视图**：描述产品所涉及的各个方面和在理想情况下最终所具有的功能
- **范围**：描述产品应包括的部分和不应包括的部分

关键要点：

1. 需求评审会议中许多问题与项目范围有关
2. 项目范围和最终目标是评审的难点和分歧焦点
3. 业务需求决定项目视图和范围
4. 功能需求必须根据用户需求考虑，且与业务需求目标一致
5. 集中关注与软件产品相关的需求
6. 业务需求定义的特性应具有相对稳定性
7. 明确定义的项目视图和范围文档可为需求变更决策提供参考

2.3.2 通过业务需求确定项目视图

- 项目视图把项目参与者定位到共同和明确的方向
- 业务需求在视图和范围上形成文档，必须在创建项目之前起草
- 业务需求从不同人那里收集，这些人了解为什么要从事该项目

2.3.3 业务需求冲突

不同渠道的业务需求可能发生冲突（如开发者、零售商、客户的目标不同），必须在软件需求说明制订之前予以解决。

2.3.4 业务需求的作用

- 决定应用程序所能实现的业务任务（用例）的设置（应用宽度）
- 决定对用例所支持的等级和深度
- 帮助确定哪个用例需要稳定综合的功能实现，哪个仅需一般实现

2.3.5 项目视图和范围文档格式

4大部分：

1. **业务需求**：背景、业务机遇、业务目标与成功标准、客户或市场需求、提供给客户的价值、业务风险
2. **视图的解决方案**：项目视图陈述、主要特性、假设与依赖环境
3. **范围与限制**：首次发行版本的范围、后续发行版本的范围、假设与排除
4. **业务背景**：涉众简介、项目优先级、操作环境

项目优先级三因素：

- **激励(driver)**：最高级别的项目目标
- **约束(constraint)**：项目管理者必须在限制因素下工作
- **自由度(degree of freedom)**：项目管理者能权衡其它方面的因素

产品成功的因素：

- 与财务有关：市场份额、毛利、维护成本等
- 与财政无关：顾客满意度、交易处理能力、客户服务响应时间等

2.4 寻找客户的需求

2.4.1 征求客户意见的步骤

1. 明确项目用户需求的来源
2. 明确使用该产品的不同类型的用户
3. 与产品不同用户类的代表进行沟通
4. 遵从项目的最终决策者的意见

> 客户参与是避免期望差异(expectation gap)的唯一途径

2.4.2 需求的来源（8个典型来源）

1. 与潜在用户进行交谈和讨论
2. 把对现有产品或竞争产品的描述写成文档
3. 系统需求规格说明
4. 现有系统的问题报告和改进要求
5. 市场调查和用户问卷调查
6. 观察用户如何工作
7. 用户工作的情景和内容分析
8. 事件和响应

2.4.3 用户类

- 每个用户类都有自己的一系列功能和非功能要求
- 用户类不一定都指人，有时应把其它应用程序或系统接口所用的硬件组件也看成附加用户类
- 用户类是产品用户的子集，产品用户是产品客户的子集，产品客户又是产品涉众的子集
- 应尽早确定并描述产品不同的用户类

2.4.4 寻找用户（产品）代表

- 每种类型项目需要挑选合适的用户代表来反映各类用户需求
- 用户代表必须参加整个软件开发生存周期
- 重点放在最重要的用户代表身上

产品代表的基本要求：

- 对新系统有明确认识并有极大热情
- 有力的交流者，且是同组成员的代表者
- 对应用领域有彻底了解
- 知道在当前技术背景下什么是可行的
- 有足够权利为代表用户作出决定

产品代表的活动：

1. 计划(planning)：推敲范围和限制、定义外部接口、评估影响、定义过渡方案
2. 验证和确认(verification and validation)：审查需求文档、确定验收标准、编写测试用例、提供测试数据、进行beta测试
3. 用户帮助(user aid)：编写用户手册和在线帮助、准备培训材料、产品演示
4. 变更管理(change management)：评估缺陷修改请求、评估改进请求、评估需求变更影响、参加变更控制委员会

应避免的问题：

1. 按授权做出的决策被经理否决
2. 产品代表只表达自己的需求
3. 缺乏对新系统充分了解而听从分析员决定
4. 资深用户推荐没有经验的用户担任产品代表
5. 用户代表试图代表不属于的用户类发表意见

第三章 需求分析

3.1 需求分析概述

3.1.1 需求分析的内容和目的

- **内容**：提炼、分析和仔细审查已收集到的需求，确保所有风险承担者明白其含义并找出错误、遗漏或其它不足
- **目的**：开发出高质量和具体的需求，以便进行实用的项目估算并进行设计、构造和测试
- 需求分析主要通过**需求建模**来实现

3.1.2 需求的描述方法

- Alan Davis：没有一种单一的需求视图能提供对需求的全面理解，必须综合使用文本和图形表示法
- 需求视图包括：功能性需求清单和表格、图形分析模型、用户界面原型、测试用例、判定树和判定表
- Karl E. Wiegers：分析模型应该是对用自然语言编写的需求规格说明的补充完善，而不是替换
- 直观的需求模型包括：数据流图(DFD)、实体-关系图(ERD)、状态转化图(STD)、对话图、用例图、类图和活动图

3.1.3 需求分析的主要任务

1. 绘制系统关联图
2. 创建用户界面（接口）原型
3. 建立数据字典
4. 为需求建立模型
5. 确定需求优先级

3.2 绘制系统关联图

3.2.1 基本概念

- **定义**：系统关联图是用于定义系统与系统外部实体间的界限和接口的简单模型，同时明确通过接口的信息流和物质流
- 关联图确定了通过某一接口与系统相连的外部实体（"端点"或"外部实体"），以及外部实体和系统之间的数据流和物流
- 通常把关联图作为数据流图的最高抽象层
- 可以把关联图写入项目视图和范围文档或SRS中

3.3 创建用户界面原型

3.3.1 用户界面原型的优势

- 当开发人员或用户不能确定需求时，开发原型使模糊问题更直观明了
- 用户通过评价原型使参与者更好理解所要解决的问题

3.3.2 使用原型的三个主要目的

1. **明确并完善需求**：原型作为需求工具，是对部分系统的初步实现
2. **探索设计选择方案**：原型作为设计工具，探索不同用户界面技术
3. **发展为最终的产品原型**：原型作为构造工具，是产品最初子集的完整功能实现

3.3.3 水平原型

- 也叫行为原型(behavioral prototype)或演示模型(mock-up)
- 像电影集，在屏幕上显示用户界面的正面像，可能允许一些导航，但仅包含少量功能
- 可以探索预期系统的特定行为，细化需求
- **注意**：水平原型中提出的功能经常并没有真正实现

3.3.4 垂直原型

- 也叫结构化原型或概念的证明
- 实现了一部分应用功能
- 用于不能确信构造方法是否完善、需要优化算法、评价数据库图表或测试临界时间需求时
- 更常用于软件设计阶段以减少风险
- 实现了客户一部分用户界面和相应的服务器功能

3.3.5 抛弃型原型

- 也称探索型原型(exploratory prototype)
- 达到预期目的后将被抛弃
- 应以最小代价尽快建立
- 不能将抛弃型原型中的代码移植到最终产品系统中
- 当遇到需求中的不确定性、二义性、不完整性或含糊性时最适合建立

利用抛弃型原型的活动序列：使用实例描述 → 对话图 → 抛弃型原型 → 详细用户界面设计

3.3.6 进化型原型

- 在已清楚定义需求的情况下，为开发渐增式产品提供坚实构造基础
- 是螺旋式软件开发生存周期模型的一部分
- 一开始就必须具有健壮性和产品质量级的代码
- 必须设计为易于升级和优化
- 适用于Web开发项目

3.3.7 原型方法对比

	抛弃型	**演化型**
水平原型	澄清并细化使用实例和功能需求；识别遗漏功能；研究用户界面方法	实现核心使用实例；根据优先级实现附加使用实例；适应快速变化的业务需求
垂直原型	证明技术可行性	实现并扩充核心客户端/服务器功能层；实现并优化核心算法；测试并调整性能

3.3.8 书面原型

- 廉价、快速、不涉及高技术的方法
- 工具仅是纸张、索引卡、粘贴纸、白板和标记器
- 方便原型的快速反复性
- 在运用自动化工具建立详细原型之前，书面原型对精化需求是优秀技术

3.3.9 原型评价

- 建立脚本使用户遵从步骤并回答特定问题
- 评价者必须是所期望用户群的代表
- 评价组必须从使用功能的用户类中挑选经验和经验不足的用户

- 通过实际观察用户使用原型获得更多信息
- 把从原型评价中获得的信息编写成文档

3.3.10 原型最大的风险

- 抛弃型原型绝不会达到产品的使用程度，仅是模型
- 处理风险承担者的期望是成功原型法的关键因素
- 决不能把抛弃型原型当作可交付的产品
- 控制风险方法：使用书面原型、使用不同于真正开发的工具

3.3.11 原型法成功的因素

1. 项目计划中应包括原型风险
2. 安排好开发、评价和修改原型的时间
3. 计划开发多个原型
4. 尽快且廉价地建立抛弃型原型
5. 抛弃型原型中不应含代码注释、输入有效性检查、保护性编码或错误处理
6. 对于已理解的需求不要建立原型
7. 不能随意增加功能
8. 不要从水平原型性能推测最终产品性能
9. 在原型中使用合理的模拟数据
10. 不要期望原型可以代替需求文档

3.4 确定需求的优先级别

3.4.1 需求优先级概述

- 以优先级为基础确定产品版本将包括哪些特性或哪类需求
- 当允许需求变更时，在特定版本中加入变更
- 设定优先级有助于项目经理解决冲突、安排阶段性交付、做出必要取舍

3.4.2 确定优先级的必要性

1. 客户期望高、开发时间短、资源有限时，必须尽早确定最重要的功能
2. 建立每个功能的相对重要性有助于规划软件构造
3. 时间盒或渐增式开发中设定优先级特别重要
4. 项目经理必须权衡合理的项目范围和约束
5. 客户必须指明哪些需求必须包括在首发版本中
6. 设定需求优先级是客户的责任之一
7. 客户和开发者都必须为设定优先级提供信息

3.4.3 设定需求优先级的方法

方法一：三级划分

命名	意义
高	关键任务需求；下一版本所需求的
中	支持必要系统操作；最终要求，可延迟到下一版本
低	功能或质量增强；资源允许时实现

方法二：基本/条件/可选

命名	意义
基本的	只有在这些需求上达成一致，软件才会被接受
条件的	增强产品性能，但忽略也可接受
可选的	实现或不实现均可

方法三：1-2-3

级别	意义
3	必须完美实现
2	需要付出努力，但不必太完美
1	可以包含缺陷

3.4.4 基于价值、成本和风险的优先级设定

优先级公式：

$$\text{优先级} = \frac{\text{价值\%}}{\text{费用\%} + \text{风险\%}}$$

其中：

- 价值% = (相对利润 × 权重$_1$ + 相对损失 × 权重$_2$) / 总价值 × 100%
- 费用% = 相对费用 / 总费用 × 100%
- 风险% = 相对风险 / 总风险 × 100%

步骤：

1. 列出所有需求/特性/使用实例（同一抽象级别）
2. 估计每个特性的相对利润（1~9）
3. 估计不实现该特性的相对损失（1~9）
4. 计算总价值 = 相对利润 + 相对损失
5. 估计每个特性的相对费用（1~9），计算费用%
6. 估计每个特性的相对风险（1~9），计算风险%
7. 用公式计算优先级
8. 按优先级降序排列

典型参与者：项目经理（指导全过程）、重要客户代表（提供受益和损失程度）、开发者代表（提供费用和风险程度）

第四章 编写需求规格说明

4.1 软件需求规格说明

4.1.1 定义

软件需求规格说明(SRS)：精确阐述一个软件系统必须提供的功能和性能以及它所要考虑的限制条件或约束。

别名：功能规格说明(Function Specification)、产品规格说明(Product Specification)、需求文档(Requirements Document)、系统规格说明(System Specification)

4.1.2 SRS的作用

- 不仅是系统规划、设计和编码的基础，也是系统测试和用户文档的基础
- 应尽可能完整地描述系统预期的外部行为和用户可视化行为
- 除设计和实现上的限制外，SRS不应包括设计、构造、测试或工程管理的细节

不同读者使用SRS的目的：

- 客户和营销部门：了解要交付什么产品
- 项目经理：制定规划、预测进度和资源
- 软件开发团队：理解将要开发的产品
- 测试小组：制定测试计划、设计测试用例
- 软件维护和支持人员：了解产品每部分功能
- 文档编写人员：编写用户手册和帮助文档
- 培训人员：编写培训材料
- 法律顾问：确保遵守法律法规
- 分包商：进行相应开发工作

4.1.3 基线

基线(Baseline)：正在开发的SRS向已通过评审的SRS的过渡过程。

- 必须通过变更控制过程来更改基线SRS
- 所有参与者必须根据已通过评审的需求安排工作
- 测试中要使用最新版本的SRS

4.1.4 需求可读性的9条建议

1. 对节、小节和单个需求的标识格式必须一致
2. 在右边部分留下文本注释区
3. 允许不加限制地使用空白
4. 正确使用各种可视化强调标志
5. 创建目录表和索引表
6. 对所有图和表进行编号并给出标题
7. 使用交叉引用功能

8. 使用超链接
9. 使用合适的模版

4.1.5 标识需求

三种标识方法：

1. **序列号**：最简单，如UR-2或SRS13；不能重用；不提供逻辑或层次区别
2. **层次型编码**：最常用，如3.2.4.3；简单紧凑但不能产生永久性标识；插入/删除会破坏引用
3. **层次化文本标签**：如Print.ConfirmCopies；结构化、有语义含义、不受增删影响；比数字标识码复杂

4.1.6 处理不完整性

- 使用"待确定"(TBD)符号标记尚未确定的需求
- 记录谁将解决哪个问题、怎样解决及什么时候解决
- 把每个TBD编号并创建TBD列表
- 在实现需求集合之前，必须解决所有TBD问题
- 如必须继续开发，尽可能推迟实现这些需求，或采取开放性设计原则

4.1.7 用户界面和SRS

消极方面：

- 屏幕图像和用户界面描述是解决方案(设计)，不是需求
- 用户界面的布局不能替代用户需求和功能需求

积极方面：

- 对可能的用户界面进行研究使需求变得实实在在
- 直观的用户界面更有助于项目计划制定和预估

4.2 软件需求规格说明模板

采用IEEE标准830-1998或ISO/IEEE 2011描述的SRS模板。

模板结构：

1. **引言**
    - 1.1 目的
    - 1.2 文档约定
    - 1.3 预期读者和阅读建议
    - 1.4 项目范围
    - 1.5 参考文献
2. **综合描述**
    - 2.1 产品的前景
    - 2.2 产品特性
    - 2.3 用户类及其特征
    - 2.4 运行环境

    - 2.5 设计和实现上的限制
    - 2.6 用户文档
    - 2.7 假设和依赖
3. **系统特性**（每个特性重复3.x.1~3.x.3）
    - 3.x.1 说明和优先级
    - 3.x.2 激励/响应序列
    - 3.x.3 功能性需求
4. **外部接口需求**
    - 4.1 用户界面
    - 4.2 硬件接口
    - 4.3 软件接口
    - 4.4 通信接口
5. **其它非功能需求**
    - 5.1 性能需求
    - 5.2 防护性需求
    - 5.3 安全性需求
    - 5.4 软件质量属性
    - 5.5 业务规则
6. **其它需求**

附录：

- 附录A：词汇表
- 附录B：分析模型
- 附录C：待确定问题的列表

4.3 编写需求文档的原则

1. 使用语法、拼写和标点正确的完整句子，保持语句和段落简短明了
2. 采用主动语态，写成"系统将做某事"
3. 使用的术语应与词汇表中所定义的一致
4. 将含糊不明的顶层需求分解成足够详细的几个需求
5. 需求说明应具有一致的风格，如"系统将……"；避免使用"应该"、"可以"、"可能"
6. 使用列表、数字、图和表来表示信息
7. 避免模糊的、主观的术语（用户友好、容易、简单、迅速、有效、健壮等）
8. 避免使用比较性词汇（提高、最大化、最小化、最佳化）
9. 定量说明所需提高的程度或参数可接受的最大值和最小值

应避免的歧义术语：可接受的、足够的、差不多、至少、最小、不多于、有效的、快的、迅速的、灵活的、健壮的、充分的、透明的、容易的、友好的、改进的、更好的、更快的、优越的、一般情况下、理性情况下、在必要的时候

文档编写注意事项：

1. 把顶层不明确的需求向低层详细分解
2. 编写单个的可测试需求文档
3. 以相同详细程度编写每个需求文档
4. 不应把多个需求集中在一个冗长的叙述段落中
5. 不应该出现需求冗余

4.4 改进前后的需求示例

例1：后台任务管理器必须在固定时间间隔内提供状态消息，且不得小于60秒

→ 改写为4条独立需求：显示状态消息区域、每60(±10)秒更新、显示完成百分比、显示"已完成"消息、显示出错消息

例2：XML编辑器必须在显示和隐藏非打印字符之间进行瞬间切换

→ 改写：用户通过特定触发机制，可在显示和隐藏所有XML标记标签之间切换，改变显示方式所需时间≤0.1秒

例3：XML解析器应该生成标记出错的报告，使初学者迅速除错

→ 改写：解析器完全解析完文件后生成出错报告（行号、文本内容、错误描述）；无错误时不生成报告

例4："如果可能的话，应当根据主要货物编号列表在线确认所输入的货物编号"

→ 改写：系统必须根据在线货物编号列表确认输入的货物编号；查不到时显示出错消息并拒绝订货

例5："产品不应该提供将带来灾难性后果的查询和替换选择"

→ 合理使用反面需求，关注焦点在于保护文件内容

4.5 基于大模型的需求规格说明示例

输入内容5类

1. 业务材料：项目视图与范围文档、业务规则、制度规范
2. 用户材料：访谈纪要、问卷、用户故事、用例
3. 系统材料：原型图、流程图、接口说明、数据字典
4. 历史材料：旧版本SRS、缺陷单、变更记录
5. 评审材料：需求编写规范、术语表、质量检查清单

典型处理流程

资料清洗与切分 → RAG检索增强 → 按模板生成结构化结果 → 自动质检与评审 → 人工确认与基线化

提示词框架要点

1. 每条需求唯一编号
2. 使用"系统必须/应"式表达
3. 避免模糊词
4. 补充异常处理
5. 对不确定内容标记TBD
6. 以表格输出"需求编号—需求内容—来源—可测试点—是否TBD"

第五章 需求验证

5.1 需求验证的提出

5.1.1 需求验证的重要性

- **验证(verification)**：判断某开发活动的工作产出是否符合要求；需求验证判断写需求的方式是否正确
- 是需求开发的最后一个环节，是一个质量关
- 需求缺陷占总缺陷的1/3，占全部返工量的70~85%
- 修复需求缺陷消耗项目预算的25~40%
- 项目交付后修复需求缺陷的成本放大60~200倍
- 沟通信息传递衰减最大值高达60%；最坏情况下开发人员获得的信息仅是原来的8.4%

5.1.2 问题避免的主要手段

1. **文档**：将沟通达成共识的信息文档化
2. **Review（审查/评审）**：通过再次审读尽早暴露错误

需求验证确保需求符合良好特征：完整的、正确的、可行的、必要的、具有优先级的、无二义性及可验证的；符合需求规格说明良好特性：完整的、一致的、易修改的、可跟踪的

V字模型：用户需求→验收测试；功能性需求→系统测试；体系结构→集成测试；详细设计→单元测试；编码

5.1.3 需求验证的主要方法

1. **评审（审查）**：对需求文档进行正式审查，验证需求是否充分正确反映用户需求
2. **编写测试用例**：根据用户需求写出黑盒功能测试用例，从测试用例追溯回功能需求
3. **编写用户手册**：在需求开发早期起草用户手册作为参考
4. **确定合格的标准**：让用户描述什么样的产品才算满足要求

5.2 需求评审

5.2.1 基础概念

- **需求评审**：让需求文档作者以外的人查验工作产物中的问题，又称"同行审查"
- 评审通过后才能作为基线发布，否则重新进行需求分析

6种评审方法（按正式程度递减）：审查 → 小组评审 → 走审 → 结对编程 → 同级评审/轮查 → 临时评审

评审员检查内容：

1. 系统定义目标是否与用户要求一致
2. 文档资料是否齐全
3. 描述是否完整、清晰、准确
4. 主要功能是否已包括在规定范围内
5. 数据流与数据结构是否确定且充分
6. 设计约束条件是否符合实际
7. 是否详细制定了检验标准

5.2.2 审查过程

1976年Michael Fagan在IBM制定，被认为是软件业最好的实践。

参与者（限制7人左右或更少）：

- 必须代表三方面观点：a)产品开发者及同组成员 b)先前产品开发者或规格说明编写者 c)根据文档确定的其他人员

角色：

1. **作者**：创建或维护被审查产品；在审查中不起积极作用
2. **调解者**：与作者制订审查计划、协调活动、推进审查会
3. **读者**：一次审查一块内容并做出解释
4. **记录员**：用标准化形式记录问题

审查阶段：

1. **规划**：决定参与者、材料、审查次数；审查速度4~6页/小时
2. **总体会议**：提供背景信息
3. **准备**：以典型缺陷清单为指导检查产品；75%的错误在此阶段发现
4. **审查会议**：读者逐条解释，记录员记录问题；不超过2小时
5. **重写**：作者修改文档
6. **重审**：确保所有问题得到解决

进入审查标准：

1. 文档符合标准模板
2. 已做拼写和语法检查
3. 作者已检查版面错误
4. 已获得参考文档
5. 打印了行序号
6. 所有未解决问题标记为TBD
7. 包括术语词汇表

退出审查标准：

1. 已明确阐述审查员提出的所有问题
2. 已正确修改文档
3. 修订文档已进行拼写和语法检查
4. 所有TBD问题已全部解决
5. 已记录审查员确认
6. 文档已登记入配置管理系统
7. 审查资料已送有关收集处

需求审查清单：

- **组织完整性**：交叉引用正确、细节一致、为设计提供足够基础、包含优先级、定义外部接口、包括所有客户需求、无遗漏信息、记录错误条件行为

- **正确性**：无冲突或重复、简明无二义、可验证、在项目范围内、无内容语法错误、可实现、错误信息唯一明确
- **质量属性**：性能目标合理、安全保密合理、折衷已记录
- **可跟踪性**：每个需求唯一可识别、可从高层需求跟踪到功能需求
- **特殊问题**：需求不是设计/实现方案、时间要求高的功能已定义标准、国际化已阐述

5.2.3 需求评审的困难

1. 大型需求文档：采用非正式渐增式审查
2. 庞大审查小组：确保每人都是为寻找错误而来
3. 审查员地域分散：使用共享网络文件夹、基于Web的聊天工具
4. 审查员准备不足：所有审查员须在正式审查前检查材料

5.2.4 基于多Agent的需求评审

8个Agent：

1. 文档解析Agent：拆解SRS结构
2. 完整性Agent：查漏项
3. 一致性Agent：查前后冲突
4. 可测试性Agent：查可测性
5. 规则验证Agent：查业务规则
6. 权限Agent：查角色边界
7. 场景Agent：造场景验证
8. 汇总Agent：汇总报告

验证方法：规则验证 + 场景验证 + 矩阵验证三结合

5.3 测试需求

5.3.1 测试需求的作用

- 以功能需求为基础或从用例派生出的测试用例可使项目参与者看清系统行为
- 设计测试用例的简单动作可以解释需求的许多问题
- 在部分需求稳定时就开始开发测试用例，可及早发现问题
- 编写黑盒测试用例可以明确特定条件下系统运行的任务
- 早期黑盒测试设计是控制项目费用和进度的强有力技术

5.3.2 测试需求案例

化学制品跟踪系统测试用例开发流程：

1. 业务需求 → 用例 → 功能需求 → 对话图 → 测试用例
2. 通过跟踪测试用例的可执行路径，发现错误或遗漏的需求
3. 对话图中未被加亮的导航线可能意味着：非法系统行为或遗漏测试用例
4. 测试用例无法执行可能意味着：测试用例错误或对话图/SRS遗漏需求

第六章 需求管理

6.1 需求管理的原则和实践

6.1.1 需求基线

需求基线(Requirement Baseline)：团队成员已承诺将在某一特定产品版本中实现的功能性和非功能性需求的一组集合。

通过正式评审和批准定义需求基线，意味着：

1. 项目涉众各方对产品功能和属性达成一致
2. 需求应进行配置管理
3. 后续变更必须遵守变更控制过程
4. 对需求文档实施版本控制

6.1.2 需求管理过程

需求管理 = 变更控制 + 版本控制 + 需求跟踪 + 需求状态跟踪

6.1.3 需求版本控制

- 每个版本必须唯一标识
- 变更必须清楚写入文档并及时通知
- 仅允许指定专人更新需求
- 每个公布版本应包括修正历史

版本号约定：

- 第一版：1.0版(草案1)
- 下一稿：1.0版(草案2)
- 被采纳：1.0正式版
- 较小修改：1.1版(草案1)
- 较大修改：2.0版

常用版本控制工具：VSS、Gitee、StarTeam、ClearCase、SVN、Git等

6.1.4 需求属性

每个需求应考虑指定的属性：

- 需求创建日期、当前版本号、创建作者、负责认可人员、拥有者/涉众列表、优先级、需求状态、稳定性
- 需求根据/来源、创建理由、涉及子系统、产品版本号、验证方法/接受标准

6.1.5 跟踪需求状态

状态值	定义
已建议	已被有权提出需求的人建议
已批准	已分析影响，已分配到基线，开发团队同意实现

状态值	定义
已实现	代码已设计、编写和单元测试，已跟踪到设计和编码元素
已验证	使用所选方法验证了实现的需求，已跟踪到测试用例，被认为完成
已删除	已从基线中删除，包括原因说明和决策人

6.1.6 评估需求管理的工作量

需求管理总工作量 = 提出变更 + 交流变更 + 评估变更 + CCB活动 + 定义跟踪信息 + 更新文档 + 跟踪和报告状态

软件配置管理(SCM)三方面内容：

1. Version Control（版本控制）
2. Change Control（变更控制）
3. Process Support（过程支持）

6.2 变更管理

6.2.1 控制项目范围的蔓延

需求蔓延：需求基线确定后又要增添新功能或进行较大改动。

- 需求蔓延高风险：80%的管理信息系统项目、70%的军事软件项目、45%的外包软件项目
- 控制方法：文档化视图/范围/限制、根据业务目标评估每项需求、原型法、敢于说"不"

6.2.2 变更控制过程

变更控制策略：

1. 所有需求变更必须遵循控制过程
2. 未获批准的变更除可行性论证外不应做其它设计和实现
3. 简单请求变更不能保证实现，由CCB决定
4. 项目风险承担者应能了解变更数据库内容
5. 绝不能从数据库中删除或修改变更请求原始文档
6. 对每个变更都应进行影响分析
7. 每个集成的需求变更必须能跟踪到经核准的变更请求
8. 记录批准或否决每个变更请求的所有理由

变更请求状态：建议者提交 → 评估者分析 → CCB决定（被拒绝/被采纳）→ 修改者实施 → 验证者确认 → 结束

6.2.3 变更控制委员会(CCB)

- 也称配置控制委员会(Configuration Control Board)
- 负责做出决定将哪些已建议的需求变更或新产品特性付诸应用
- 大项目可有多级CCB
- 成员应能代表变更涉及的团体

CCB角色：

- CCB主席：意见不一致时可独自决定，选定评估者和修改者
- CCB：决定采纳或拒绝变更请求
- 评估者：分析变更影响
- 修改者：实现已认可的变更
- 提议者：提交变更请求
- 请求接受者：接受提交的变更请求
- 验证者：决定变更是否已正确执行

CCB决策权衡：

- 利：节省资金/额外收入、增强客户满意度、竞争优势、减少上市时间
- 弊：增加开发费用、推迟交付日期、产品质量下降、减少功能、用户不满意

6.2.4 变更控制工具

挑选工具考虑方面：

1. 可定义变更请求数据项
2. 可定义状态转换图
3. 可加强状态转换
4. 记录状态变更数据
5. 可定义自动通知
6. 可生成报告和图表
7. 内置变更建议系统

6.2.5 测量变更活动

值得考虑的方面：

- 接收、未作决定、结束处理的变更请求数量
- 已实现需求变更合计数量
- 每个方面发出的变更请求数量
- 每个已应用需求建议变更和实现变更的数量
- 投入处理变更的人力物力

6.3 需求变更影响分析

6.3.1 影响分析过程

表A：变更涉及的问题核对表：

- 基线中是否有需求与建议变更冲突？
- 是否有待解决的需求变更与建议变更冲突？
- 不采纳变更有什么业务或技术后果？
- 建议变更有什么负面效应或风险？
- 变更是否不利于其它质量属性？
- 从技术条件和员工技能看是否可行？

- 变更是否影响市场营销、制造、培训或用户支持计划?
- 实现变更是否提出不合理要求?
- 是否有额外工具要求?
- 如何影响任务执行顺序、依赖性、工作量或进度?
- 是否要求原型法?
- 浪费了多少以前曾做的工作?
- 是否导致产品单元成本增加?

表B：变更影响的软件元素核对表:

- 用户接口变更、报告/数据库/文件变更
- 设计部件变更、源代码变更
- 测试用例修改/新增、应用软件/库/硬件部件变更
- 第三方软件、项目管理计划变更
- 帮助文件/培训教材/用户文档变更

评估流程:

1. 按表A进行
2. 按表B进行，使用跟踪能力信息
3. 使用劳动时数表评估工作量
4. 求评估工作值总和
5. 确认任务执行顺序
6. 决定变更是否处于临界路径
7. 估计变更如何影响进度和费用
8. 评估变更优先级
9. 向CCB报告影响分析结果

6.3.2 影响分析报告模板

包含：变更请求ID、标题、分析者、日期、描述、优先权评估（相关收益/代价/成本/风险1-9）、计算优先级、预计总工作量、预计损失工作量、预计对进度影响、额外成本影响、质量影响、被影响的其他需求/任务、要更新的计划、集成问题、生存期成本事项、需检查的其他部件

6.4 需求跟踪

6.4.1 需求跟踪定义

需求跟踪：编制每个需求同系统元素之间的联系文档。这些元素包括别的需求、体系结构、设计部件、源代码模块、测试、帮助文件、文档等。

跟踪能力链(traceability link)：使我们能跟踪一个需求使用期限的全过程，即从需求源到实现的前后生存期。

四类需求跟踪能力链:

1. 向前追溯到需求
2. 回溯到需求
3. 向前跟踪到下游产品

4. 向后追溯到需求

6.4.2 需求跟踪的好处

1. **审核(certification)**：确保所有需求被应用
2. **影响分析**：确保不忽略受影响的系统元素
3. **维护**：正确完整实施变更，提高生产率
4. **项目跟踪**：获得功能当前实现状态记录
5. **再设计**：记录传统系统功能在新系统中的位置
6. **重用**：在新系统中利用旧系统相关资源
7. **减小风险**：文档化部件互连关系减少关键成员离开的风险
8. **测试**：联系链在测试出错时指出最可能有问题的代码段

6.4.3 需求跟踪方式

需求跟踪能力矩阵：

使用实例	功能需求	设计元素	代码	测试实例
UC-28	Catalog.query.sort	Class catalog Search.7	Catalog.sort()	Search.8
UC-29	Catalog.query.import	Class catalog Search.12	Catalog.import()	Search.13

联系链类型：

- 一对一：一个代码模块实现一个设计元素
- 一对多：多个测试实例验证一个功能需求
- 多对多：每个用例导致多个功能性需求

跟踪联系链信息源：

源对象类型	目的对象类型	信息源
系统需求	软件需求	系统工程师
用例或业务规则	功能性需求	需求分析员
功能性需求	功能性需求	需求分析员
功能性需求	软件体系结构元素	软件架构师
功能性需求	其他设计元素	设计人员/开发人员
设计元素	代码	开发人员
功能性需求	测试用例	测试工程师

6.5 需求管理工具

6.5.1 基于文档存储需求的限制

1. 很难保持文档与现实一致
2. 通知受变更影响的设计人员是手工过程
3. 不太容易为每个需求保存增补信息
4. 很难在功能需求与用例、设计、代码、测试之间建立联系链
5. 很难跟踪每个需求的状态
6. 很难同时管理多个产品版本的需求集
7. 多人修改需求困难
8. 没有合适地方存储被否决/已删除的需求

6.5.2 商业需求管理工具

工具	供应商	存储方式
CaliberRM	Borland/Micro Focus	数据库
DOORS	Telelogic	数据库
PingCode	Worktile	数据库
RequisitePro	Rational	文档
RTM Workshop	Integrated Chipware	数据库
oKit	Qingflow	数据库
RMTrak	RBC	文档
Ones	Ones Software	数据库
Vital Link	Compliance Automation	文档

6.5.3 使用需求管理工具的益处

1. 管理版本和变更
2. 存储需求属性
3. 进行影响分析
4. 跟踪需求状态
5. 访问控制
6. 与涉众进行沟通
7. 重用需求

6.5.4 需求管理工具的功能

- 定义不同种类的数据库元素
- 与Word集成
- 层次编码数字标签
- 层次显示方法
- 文档加工功能
- 跟踪能力联系链
- 用户权限管理
- 与其他开发工具集成

6.5.5 选择工具的过程

1. 定义组织对需求管理工具的需求
2. 列出影响决策的10-15个因素并打分
3. 获得可用工具最新信息，对候选工具排序
4. 计算每个候选工具得分
5. 从其他用户获得体会
6. 从前三名开发商获得评估拷贝
7. 用实际项目评估工具
8. 综合考虑做出决定